

GPU Concurrent Real-time Ray tracing engine

Taras Levytskyi

Mentors: Ivan Shevchuk, Danyil Bykov, Ostap Trush

Applied Sciences Faculty

Ukrainian Catholic University

Lviv, Ukraine

llevttarr@gmail.com

Abstract—This report presents KHORS (Kinematic Hybrid Optimized Ray System), a real-time ray-tracing 3D engine implemented using the C++ language, OpenGL/Vulkan API and GPU compute shaders. The engine renders scenes by casting rays from the camera into the scene and computing intersections with primitives to determine pixel colors.

I. INTRODUCTION

This document is a model and instructions for L^AT_EX. Please observe the conference page limits.

II. SYSTEM OVERVIEW

KHORS (Kinematic Hybrid Optimized Ray System) is a ray-tracing 3D engine that uses the Vulkan API and ray-tracing techniques to calculate the resulting screen image.

III. SOFTWARE DESIGN

There are 2 main ways for 3D engines to show objects on a 2D screen. The first is rasterizing - by saving primitive positions and other relevant data into GPU buffers, and then outright calculating the screen pixels. While it is the fastest method, it does not reflect the way light works in real life. The other approach to 3D rendering, used in this project, is ray tracing.

By 'shooting' rays from the screen to the scene, we can calculate fragment colors by finding intersections of the rays with the objects on the scene. The primary ray equation is:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$$

where $\mathbf{o}$ is the vector representing the origin position of the vector, $\mathbf{d}$ is the vector representing the direction of the ray, and t is the variable representing at which point does the ray intersect a primitive - if it does.

The calculations are run through a GPU rendering pipeline system, which consists of 4 compute stages, two of which cast rays (Sampling and Shading) and are split into a few shaders based on ray calculation stage; the other two (Temporal and Spatial reuse) update the sampling data structures (see below). Memory allocation of buffers happens once per scene rebuilding. With this in mind, the pipeline looks like this:

The foundation of rendering is the rendering equation:

$$L_0(\vec{x}, w_0) = L_e(\vec{x}, w_0) + \int_{\Omega} f_r(\vec{x}, w_i, w_0) L_i(\vec{x}, w_i) \vec{n} \cdot \vec{w}_i \cdot dw_i$$

The equation expresses the outgoing radiance at a point as the sum of emitted light and an integral over all incoming

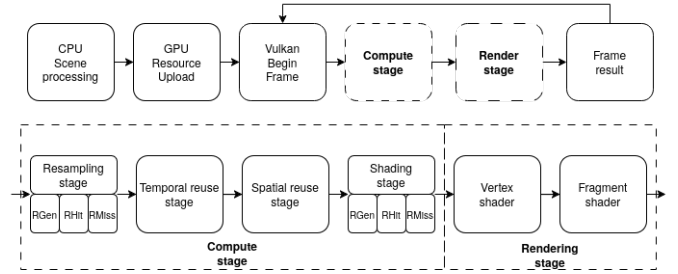


Fig. 1. KHORS pipeline

directions. The integrand consists of the BRDF, incoming radiance, and a cosine term. In practice, direct evaluation leads to a linear increase in shadow ray computations with respect to the number of light sources. To address this, the ReSTIR DI (Reservoir Spatiotemporal Importance Resampling: Direct Illumination) algorithm, developed by NVIDIA, is used.

Explicit evaluation is replaced with a sampling-based approach using reservoir sampling and importance weighting. For each point, a small set of light candidates is sampled randomly. Each candidate's contribution is evaluated based on the BRDF, geometry and light intensity. A reservoir sampling scheme is then used to retain a single candidate.

Temporal and spatial reuse are applied based on two assumptions:

- 1) Neighbouring pixels tend to have similar lighting
- 2) Lighting changes slowly over time.

After the final light candidate is chosen, we cast only one ray towards the selected light. The estimator used in this method is unbiased, so its expected value converges to the actual radiance.

For the optimization of ray calculation, the BVH (Bounding Volume Hierarchy) data structure is used. During ray traversal, the shader first tests intersections against large AABBs (axis-aligned bounding boxes), and only traverses deeper into the tree if an intersection is found. In hardware-accelerated ray tracing, BVH traversal and ray-triangle intersection are performed directly by RT cores - specialized hardware units made solely for the purpose of acceleration of ray tracing.